

HashSet with Open Addressing

Open Addressing, Linear Probing, Tombstones, Rehashing · Mentor-Led Lab

Developed by TalenciaGlobal

Difficulty ★★★ Intermediate-Advanced

Problem

Design a MyHashSet of integer keys using open addressing - all entries live in one array, and collisions are resolved by probing for the next slot rather than chaining. Implement linear probing fully; quadratic probing and double hashing are discussed as extensions.

Support add(key) (ignore duplicates), contains(key), and remove(key). Deletion uses tombstones (a deleted marker) so that probe sequences are not broken. Keep the load factor at or below 0.7, rehashing to roughly double the capacity when it is exceeded.

Learning objectives: weigh chaining against open addressing, master linear probing, handle deletion correctly with tombstones, and trigger rehashing as the table fills.

Operations & Rules

- ADD key - insert the key; if it already exists, do nothing.
- CONTAINS key - output true or false.
- REMOVE key - if present, replace the key with a tombstone (not an empty slot).
- Slots are marked empty (-1) or tombstone (-2); linear probing visits $(\text{hash} + i) \bmod \text{capacity}$ for $i = 0, 1, 2, \dots$
- Load factor: rehash to double the capacity whenever $\text{size} / \text{capacity}$ exceeds 0.7 after an insertion.

Input / Output Format

Input: Line 1: the initial capacity. Each later line: ADD key, CONTAINS key, or REMOVE key (read until end of input).

Output: One line per CONTAINS (true/false), then Size and Capacity of the final table.

Examples

Example 1

Input: cap=8; add(1); add(9); add(17); contains(9); remove(9); contains(9); add(9)

Output:

```
true  
false  
true
```

Explanation: 1, 9 and 17 all hash to bucket 1 and probe to 1, 2, 3; after removing 9 a tombstone marks its slot, then re-adding 9 reuses it.

Example 2

Input: cap=8; add(5); contains(5); remove(5); contains(5)

Output:

```
true  
false
```

Explanation: 5 is present, then absent once removed.

Constraints

- $0 \leq \text{key} \leq 10^6$; up to 10,000 operations.
- Open addressing only - no built-in set types.
- Use tombstones for deletion; rehash keeps the load factor ≤ 0.7 .